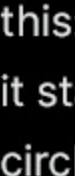
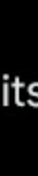




ralph for idiots: the only explanation you need



agrim singh 
@agrimsingh · 11 Oct

Takip et

🗨 44

🔁 144

❤ 18

👤 435.8

🔖

🔗

everyone on my timeline is “ralph-pilled” right now.

but if you’ve ever let an ai coding session run for 40–60 minutes, you’ve felt this:
it starts repeating itself, undoing its own fixes, and confidently going in circles.

most explanations of ralph are either:

- terminal priestcraft, or
- “it’s just a loop lol” (true, but missing the point)

@GeoffreyHuntley coined and popularized the technique and wrote the canonical post. start there if you want the origin story and the philosophical framing.

this is the 5-minute, copy-paste, no-mysticism version.

1) ralph is not “an agent that remembers forever”

ralph is the opposite: it’s an agent that **forgets on purpose**.

in geoff’s purest description, ralph is literally a bash loop that keeps starting over with fresh context:

```
bash
while :; do cat prompt.md | agent; done
```

same task. new brain each iteration.

the “memory” is not the chat. it’s the **filesystem + git**.

if it’s not written to a file, it doesn’t exist.

2) the only insight that matters: context pollution

every ai coding session has a context window (working memory). stuff goes in:

- files it read
- commands it ran
- outputs it produced
- wrong turns it took
- half-baked plans it hallucinated at 2:13am

here’s the cursed part: **you can keep adding, but you can’t delete**.

failures accumulate like plaque. eventually you hit the familiar symptom cluster:

- repeating itself
- “fixing” the same bug in slightly different ways
- confidently undoing its own previous fix
- circular reasoning, but with commit rights

that’s context pollution. once you’re there, **“try harder” doesn’t work**.

adding more instructions doesn’t help. more tokens don’t help. more patience doesn’t help.

once the ball is in the gutter, adding spin doesn’t save it.

ralph doesn’t try to clean the memory. it throws it away and starts fresh.

3) if you rotate constantly, how do you make progress?

you **externalize state**.

the trick is simple:

progress persists. failures don’t.

context (bad for state)

- dies with the convo
- persists forever
- polluted by dead ends

files + git (good for state)

- only what you choose to write
- can’t be edited
- can be patched / rolled back “memory” can drift
- git doesn’t hallucinate

each fresh agent starts clean, then reconstructs reality from files.

4) the anchor file (source of truth)

every ralph setup needs a single source-of-truth file that survives rotations and tells a brand-new agent what reality currently looks like.

in my cursor implementation, that file is `ralph_task.md`:

```
markdown
---
task: build a rest api
test_command: "npm test"
---

# task: rest api

## success criteria
1. [ ] get /health returns 200
2. [ ] post /users creates a user
3. [ ] all tests pass
```

state lives in `.ralph/`:

what are the other files and their purpose in running ralph correctly?

- guardrails.md**: learned constraints (“signs”)
- progress.md**: what’s done / what’s next
- errors.log**: what blew up
- activity.log**: tool usage + token tracking

the loop reads these every iteration.

fresh context. persistent state.

the loop is not the technique. state hygiene is the technique.

format doesn’t matter. invariants do.

5) why the claude code plugin approach is accidentally anti-ralph

let me be explicit: **the claude code plugin approach is accidentally anti-ralph**.

it keeps pounding the model in a single session until context rots. the session grows until it falls apart. no visibility into context health. no deliberate rotation.

claude code treats context rot as an accident.

ralph treats it as a certainty.

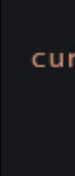
ralph solves this by **starting fresh sessions before pollution builds up**. deliberate rotation, not accidental compaction.

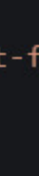
the claude code plugin lets a single session grow until it inevitably rots, with no real visibility into when context has gone bad. ralph assumes pollution is coming and **rotates deliberately before it happens**.

instead of repeating the same mistakes over and over, ralph **records failures as guardrails** so they don’t recur.

and while claude code locks you into a single model, ralph-technique should be flexible enough for you to use the right model for the job as conditions change.

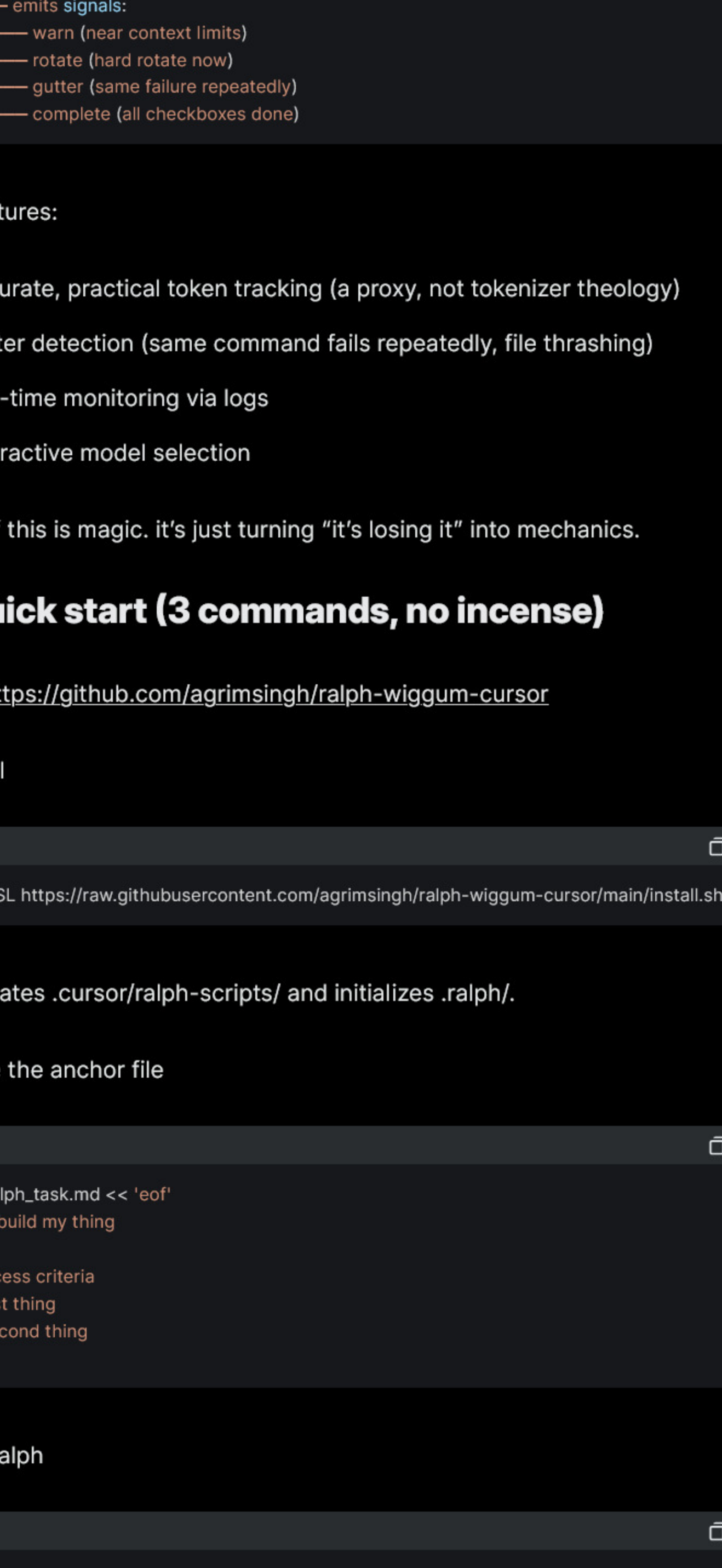
6) why i built a cursor port (model selection matters)



agrim singh 
@agrimsingh · 4 Oct

fatidfully built ralph wiggum for @cursor_ai cli. let it make mistakes, add signs. tune it like a guitar until it plays the right notes.

context is memory
malloc() exists, free() doesn't
ralph is just accepting that reality
@GeoffreyHuntley @leerob @ericzakariasson
demo +
Daha fazla göster



🗨 15

🔁 22

❤ 221

🔖

🔗

i built this because cursor lets you extend the agent loop like a real system (scripts, parsers, signals), and because **model choice matters in practice**.

different models fail in different ways. ralph lets you exploit that instead of being stuck with one failure mode.

cursor makes it trivial to swap models per iteration. different brains for different failure modes. this is deeply under-discussed compared to “one agent to rule them all.”

practical guidance:

- starting a new project → opus (architecture matters)
- stuck on something weird → codex

i’m getting better results on some workloads with gpt-codex models than opus 4.5. vibes? tokenization? inductive bias? the gods? idk. but it’s repeatable.

and yes, i’ve used this to port very large repos (tens of thousands of loc) to typescript without it faceplanting every 10 minutes. that’s the whole point: long-running implementation work where humans become the bottleneck.

7) the architecture (cursor version)

(if you don’t care about plumbing, you can skip this section. the only point is that vibes get turned into signals.)

```
css
ralph-setup.sh (interactive ui, model picker)
├──
├── cursor-agent --output-format stream-json
├──
├── stream-parser.sh (parses output, tracks usage, detects patterns)
├──
├──   ├── writes to .ralph/activity.log
├──   ├── writes to .ralph/errors.log
├──   └── emits signals:
├──       ├── warn (near context limits)
├──       ├── rotate (hard rotate now)
├──       ├── gutter (same failure repeatedly)
├──       └── complete (all checkboxes done)
```

key features:

- accurate, practical token tracking (a proxy, not tokenizer theology)
- gutter detection (same command fails repeatedly, file thrashing)
- real-time monitoring via logs
- interactive model selection

none of this is magic. it’s just turning “it’s losing it” into mechanics.

8) quick start (3 commands, no incense)

repo: <https://github.com/agrimsingh/ralph-wiggum-cursor>

1) install

```
bash
curl -fsSL https://raw.githubusercontent.com/agrimsingh/ralph-wiggum-cursor/main/install.sh | t
```

this creates `.cursor/ralph-scripts/` and initializes `.ralph/`.

2) write the anchor file

```
bash
cat > ralph_task.md << 'eof'
# task: build my thing

## success criteria
1. [ ] first thing
2. [ ] second thing
eof
```

3) run ralph

```
bash
./cursor/ralph-scripts/ralph-setup.sh
```

optional: watch it like it’s a fish tank.

```
bash
tail -f .ralph/activity.log
```

9) guardrails: how ralph stops repeating the same dumb mistake

ralph will do something stupid. the win condition is not “no mistakes.”

the win condition is **the same mistake never happens twice**.

when something breaks, the agent adds a sign to `.ralph/guardrails.md`:

```
markdown
### sign: check imports before adding
- trigger: adding a new import statement
- instruction: check if import already exists
- added after: iteration 3 (duplicate import broke build)
```

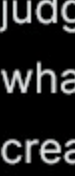
guardrails are append-only. mistakes evaporate. lessons accumulate.

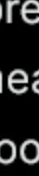
next iteration reads guardrails first. cheap. brutal. effective.

it’s basically kaizen, but for a golden retriever with a soldering iron.

10) “isn’t this just slop?”

saw this tweet earlier:



James Long 
@jlongster · 10 Oct

I haven't actually look at the ralph stuff

something about it just feels wrong to me. like a new level of slop by just getting whatever works

I still don't understand why you don't want to be deeply involved in the creative tech process

🗨 48

🔁 9

❤ 224

🔖

🔗

fair concern.

there are two modes of development:

- exploration** — figuring out what to build, experimenting, making architectural decisions
- implementation** — building the thing you’ve already designed

ralph is for #2.

if you’re exploring, use interactive mode. be deeply involved. make creative decisions.

but once you know what you’re building - a rest api with these endpoints, a cli with these commands, tests for these functions - that’s implementation. that’s ralph territory.

“but won’t it produce slop?”

only if you let it.

ralph has:

- checkboxes (explicit success criteria)
- tests (code must pass)
- types (errors get caught)
- guardrails (failures don’t repeat)
- git review (you still review everything)

ralph with proper feedback loops produces more consistent code than a tired developer at 2am.

“why wouldn’t i want to be involved?”

you ARE involved. your role just changes.

- you define what “done” means
- trigger: adding a new import statement
- you review outcomes, not keystrokes
- you decide when to intervene

think of it as **steering, not rowing**.

11) when NOT to use ralph

ralph is for **implementation, not exploration**.

use ralph when the specs are crisp, success is machine-verifiable (tests, types, lint), and the work is bulk execution like crud, migrations, refactors, or porting. it shines when you can clearly define “done” and express it as checkboxes, then let the loop grind through implementation without losing the plot.

don’t use ralph when you’re still deciding what to build, when taste and judgment matter more than correctness, or when you can’t cleanly define what “done” even means. if the real work is thinking, exploring, or making creative decisions, looping is the wrong tool - that’s interactive territory.

if you can’t write checkboxes, you’re not ready to loop. you’re ready to think.

12) the one-liner takeaway

ralph works because **it treats ai like a volatile process, not a reliable collaborator**.

your progress should persist. your failures should evaporate.

everything else - loops, scripts, signals - is just furniture around that idea.

Kendri Makalemi yayınlamak mı istiyorsun?

Premium kademesine yükselt

agrim singh
@agrimsingh

building the data layer for electronics manufacturing @ niyam.xyz // code w/ ai: codewithai.xyz // ambassador @cursor_ai @v0 // whisky guru and dj

Takip et